

REST - Get country based on country code

country.xml:

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
       http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="countryList" class="java.util.ArrayList">

        <constructor-arg>

            <list>

                <bean class="com.cognizant.spring_learn.model.Country">

                    <property name="code" value="IN" />

                    <property name="name" value="India" />

                </bean>

                <bean class="com.cognizant.spring_learn.model.Country">

                    <property name="code" value="US" />

                    <property name="name" value="United States" />

                </bean>

                <bean class="com.cognizant.spring_learn.model.Country">

                    <property name="code" value="DE" />

                    <property name="name" value="Germany" />

                </bean>

                <bean class="com.cognizant.spring_learn.model.Country">

                    <property name="code" value="JP" />
```

```
        <property name="name" value="Japan" />
    </bean>
</list>

</constructor-arg>

</bean>

</beans>
```

SpringLearnApplication.java:

```
package com.cognizant.spring_learn;
```

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.ImportResource;
```

```
@SpringBootApplication
```

```
@ImportResource({"classpath:date-format.xml", "classpath:country.xml"})
```

```
public class SpringLearnApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringLearnApplication.class, args);
    }
}
```

CountryService.java:

```
package com.cognizant.spring_learn.service;
```

```
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.ApplicationContext;
```

```
import org.springframework.stereotype.Service;
import com.cognizant.spring_learn.model.Country;

@Service

public class CountryService {

    @Autowired
    private ApplicationContext context;

    @SuppressWarnings("unchecked")
    public Country getCountry(String code) {
        // Retrieve the countryList bean from context
        List<Country> countries = context.getBean("countryList", List.class);

        // Filter through the list matching code case-insensitively
        return countries.stream()
            .filter(country -> country.getCode().equalsIgnoreCase(code))
            .findFirst()
            .orElse(null);
    }
}
```

CountryController.java:

```
package com.cognizant.spring_learn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.ApplicationContext;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.spring_learn.model.Country;
import com.cognizant.spring_learn.service.CountryService;
```

```
@RestController
```

```
public class CountryController {
```

```
    private static final Logger LOGGER = LoggerFactory.getLogger(CountryController.class);
```

```
    @Autowired
```

```
    private ApplicationContext context;
```

```
    @Autowired
```

```
    private CountryService countryService;
```

```
    // Existing endpoint from previous milestone
```

```
    @RequestMapping(value = "/country", method = RequestMethod.GET)
```

```
    public Country getCountryIndia() {
```

```
        LOGGER.info("--- START: Entering getCountryIndia() ---");
```

```
        Country india = context.getBean("india", Country.class);
```

```
        LOGGER.info("--- END: Exiting getCountryIndia() ---");
```

```
        return india;
    }

    // New Endpoint: Get country based on path variable code
    @GetMapping("/countries/{code}")
    public Country getCountry(@PathVariable String code) {
        LOGGER.info("--- START: Entering getCountry() for code: {} ---", code);

        Country country = countryService.getCountry(code);

        LOGGER.info("--- END: Exiting getCountry() ---");

        return country;
    }
}
```

Output:

The screenshot shows the Postman interface for a GET request to `http://localhost:8083/countries/in`. The request is successful, returning a 200 OK status with a response time of 40 ms and a body size of 184 B. The response body is in JSON format, containing the following data:

```
{  "code": "IN",  "name": "India"}
```

Key	Value	Description
Postman-Token	<calculated when request is sent>	
Host	<calculated when request is sent>	
User-Agent	PostmanRuntime/7.54.0	
Accept	*/*	
Accept-Encoding	gzip, deflate, br	
Connection	keep-alive	

The screenshot shows the Chrome DevTools interface. The Network panel displays the request details for `http://localhost:8083/countries/in`, including the status code (200 OK), response headers, and request headers. The Console panel shows the JSON response:

```
{  "code": "IN",  "name": "India"}
```

Request Details:

- Request URL:** `http://localhost:8083/countries/in`
- Request Method:** GET
- Status Code:** 200 OK
- Remote Address:** `[::1]:8083`
- Referrer Policy:** `strict-origin-when-cross-origin`

Response Headers:

- Connection:** keep-alive
- Content-Length:** 28
- Content-Type:** application/json
- Date:** Mon, 29 Jun 2026 15:34:27 GMT
- Keep-Alive:** timeout=60

Request Headers:

- Accept:** `text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8`
- Accept-Encoding:** `gzip, deflate, br, zstd`
- Accept-Language:** `en-US,en;q=0.5`
- Cache-Control:** `max-age=0`
- Connection:** keep-alive
- Cookie:** `csrfToken=TXWyoRQ20H@RySntLdmrSGf7c7DDdkIO`
- Host:** `localhost:8083`